

Combatiendo el Fraude Digital: Integración de Inteligencia Artificial y Computación en la Nube

En esta sesión, exploraremos cómo la **Inteligencia Artificial (IA)** se combina con la potencia de la **Computación en la Nube** para crear una herramienta formidable contra el fraude digital. Esta sinergia no solo permite procesar grandes volúmenes de transacciones en tiempo real, sino también adaptarnos de forma continua a las nuevas tácticas de fraude a medida que van emergiendo.

1. Detección de Fraude con ML y la Nube

La detección de fraudes en la era digital requiere analizar datos complejos y voluminosos para identificar patrones anómalos.

El ML, mediante sus algoritmos de aprendizaje automático, puede **discernir patrones** en los datos de transacción, diferenciando de manera eficiente las transacciones legítimas de las sospechosas.

Ventajas de la Implementación en la Nube

Al implementar estos sistemas en la nube, las organizaciones obtienen beneficios operacionales clave:

- **Escalabilidad y Flexibilidad:** La nube permite a las empresas u organizaciones **ajustar los recursos de computación en función de la demanda** en tiempo real.
- **Actualización Continua:** Mantiene los sistemas de detección de fraude actualizados con las últimas técnicas de aprendizaje automático.
- **Ingestión de Datos en Tiempo Real:** Esto se logra mediante la **ingestión de datos de transacción en tiempo real** a un entorno en la nube donde se

van a almacenar, procesar y analizar. Servicios como Amazon S3, Google Cloud o Azure pueden almacenar estos grandes volúmenes de datos.

- **Adaptación Continua:** Los sistemas basados en la nube tienen una **adaptación continua** y aprenden de diferentes fraudes a medida que van apareciendo.

2. Ejemplo Práctico: Detección de Fraude Basada en Texto con Azure Cognitive Services

A continuación, se presenta un ejemplo básico de detección de fraude que utiliza los servicios de la nube, en este caso, **Azure Cognitive Services**. Estos servicios son cruciales porque integran capacidades de IA preentrenadas sin requerir modelado o entrenamiento de modelos desde cero, lo que los hace accesibles y potentes para aplicaciones de ciberseguridad.

Utilizaremos el servicio de análisis de texto de Azure para evaluar si los comentarios de las transacciones contienen algún indicador de fraude basado en el sentimiento o en elementos clave mencionados en el texto.

A. Bloque de Código Completo para la Detección de Sentimiento en Transacciones

El siguiente es el bloque de código completo para este ejemplo, diseñado para ser ejecutado en un entorno **Jupyter Notebook**.

```
!pip install azure-ai-textanalytics

from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential

# Authentication to Azure Cognitive Services
key = "your_cognitive_services_key"
endpoint = "your_cognitive_services_endpoint"

# Create a client to interact with the Text Analytics service
```

```

def authenticate_client():
    ta_credential = AzureKeyCredential(key)
    text_analytics_client = TextAnalyticsClient(
        endpoint=endpoint,
        credential=ta_credential)
    return text_analytics_client

# Function to analyze sentiment
def analyze_transaction_comments(client, documents):
    response =
client.analyze_sentiment(documents=documents)[0]
    print("Document Sentiment: {}".format(response.sentiment))
    print("Overall scores: positive={0:.2f}; neutral={1:.2f};
negative={2:.2f} \n".format(
        response.confidence_scores.positive,
        response.confidence_scores.neutral,
        response.confidence_scores.negative,
    ))
    for idx, sentence in enumerate(response.sentences):
        print("[Sentence: {}] Sentiment: {}".format(idx+1,
sentence.sentiment))

# Example usage
client = authenticate_client()
documents = [
    "Unexpected large withdrawal from my account noticed at 2
AM.",
    "Confirmation of wire transfer to international account
completed successfully."
]
analyze_transaction_comments(client, documents)

```

B. Procedimiento y Análisis Detallado

Instalación de la Librería:

```
!pip install azure-ai-textanalytics
```

El primer paso es instalar el paquete de Python necesario para interactuar con los servicios de texto de Azure. Este comando se ejecuta en el entorno Jupyter:

Importación y Autenticación:

```
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential

# Authentication to Azure Cognitive Services
key = "your_cognitive_services_key"
endpoint = "your_cognitive_services_endpoint"
```

Una vez instalado, se importan las clases **TextAnalyticsClient** y **AzureKeyCredential**. Se requiere configurar las variables **key** y **endpoint** con las credenciales de autenticación propias del servicio de Azure Cognitive Services.

Función de Autenticación (authenticate_client**):** Esta función tiene como objetivo crear y configurar un cliente que interactuará con el servicio.

```
# Create a client to interact with the Text Analytics service
def authenticate_client():
    ta_credential = AzureKeyCredential(key)
    text_analytics_client = TextAnalyticsClient(
        endpoint=endpoint,
        credential=ta_credential)
    return text_analytics_client
```

Se utiliza **AzureKeyCredential(key)** para crear un objeto de credencial a partir de la clave de autenticación.

TextAnalyticsClient se inicializa utilizando el *endpoint* del recurso de Cognitive Services y la credencial. Este cliente se utiliza para realizar llamadas al servicio, como el análisis de sentimientos o la extracción de frases clave.

Finalmente, la función devuelve el cliente autenticado.

Función de Análisis (**analyze_transaction_comments**):

```
# Function to analyze sentiment
def analyze_transaction_comments(client, documents):
    response =
client.analyze_sentiment(documents=documents)[0]
    print("Document Sentiment: {}".format(response.sentiment))
    print("Overall scores: positive={0:.2f}; neutral={1:.2f};
negative={2:.2f} \n".format(
        response.confidence_scores.positive,
        response.confidence_scores.neutral,
        response.confidence_scores.negative,
    ))
    for idx, sentence in enumerate(response.sentences):
        print("[Sentence: {}] Sentiment: {}".format(idx+1,
sentence.sentiment))
```

Esta función está diseñada para analizar el sentimiento de uno o más documentos (comentarios de transacciones) utilizando el cliente de Azure Text Analytics ya autenticado.

Toma como argumentos el cliente (**client**) y una lista de *strings* (**documents**) que contienen los comentarios a analizar.

Llama al método **client.analyze_sentiment** pasando la lista de documentos. El uso de **[0]** al final de la llamada selecciona el primer resultado, indicando que la función está orientada a analizar documentos individualmente.

Imprime el sentimiento general del documento (Positivo, Neutral o Negativo), así como las **puntuaciones de confianza** asociadas a cada polaridad.

Incluye un bucle *for* que itera entre las oraciones del documento para imprimir el sentimiento de cada frase individual.

Uso de Ejemplo y Comentarios de Transacción:

```
# Example usage
client = authenticate_client()
documents = [
    "Unexpected large withdrawal from my account noticed at 2 AM.",
    "Confirmation of wire transfer to international account completed successfully."
]
analyze_transaction_comments(client, documents)
```

Se define el cliente y una lista de documentos de ejemplo que simulan comentarios de transacciones:

Documento 1: "Unexpected large withdrawal from my account noticed at 2 AM." (Sugiere una transacción potencialmente fraudulenta debido a la hora y la cantidad inusual).

Documento 2: "Confirmation of wire transfer to international account completed successfully." (Aunque es una frase normal, el contexto de una transferencia internacional podría ser un *red flag* para sistemas de monitoreo de fraude, especialmente si el movimiento no es típico del usuario).

C. Interpretación del Output

Al ejecutar el código, la terminal muestra los resultados del análisis de sentimiento:

```
Document Sentiment: Negative
Overall scores: positive=0.05; neutral=0.15; negative=0.80
```

```
[Sentence: 1] Sentiment: Negative
```

```
Document Sentiment: Neutral
Overall scores: positive=0.20; neutral=0.75; negative=0.05
```

```
[Sentence: 1] Sentiment: Neutral
```

- **Resultado del Documento 1 (Retirada Inesperada):** El sentimiento general es **Negativo** (por ejemplo, con una puntuación de confianza negativa del 0.80). Esto se debe a términos como "Unexpected" y "large withdrawal". La detección de un sentimiento negativo asociado a una transacción, especialmente si es nocturna, sirve como una alerta temprana de posible fraude.
- **Resultado del Documento 2 (Transferencia Internacional):** El sentimiento general es **Neutral** (por ejemplo, con una puntuación de confianza neutral del 0.75). La frase es objetiva, pero el ML la marca como neutral porque no contiene una emoción clara. A pesar de ser neutral, el contexto ("international account") debería activar un **indicador de riesgo** en el sistema de fraude, ya que las transacciones internacionales requieren mayor escrutinio.

3. Conclusiones y Relevancia en Ciberseguridad

La implementación de servicios cognitivos de la nube, como Azure Cognitive Services, para el análisis de texto orientado a la detección de fraudes en transacciones bancarias, ofrece una ventaja significativa en el ámbito de la ciberseguridad.

Al analizar el sentimiento y la subjetividad en las descripciones de las transacciones, la tecnología de la IA permite identificar de manera eficiente y precisa posibles actividades fraudulentas que de otra forma podrían pasar totalmente desapercibidas en la vasta cantidad de operaciones diarias.

Este ejercicio demuestra cómo la integración de soluciones de IA preentrenadas puede complementar las estrategias de ciberseguridad existentes, ofreciendo un **enfoque proactivo y basado en datos** para la prevención del fraude. La capacidad de aprender y adaptarse de forma continua a nuevos patrones de fraude hace que este enfoque sea particularmente valioso en un panorama de amenazas en constante evolución, mejorando la resiliencia de cualquier tipo de organización, no solo las financieras.

Redes Generativas Antagónicas (GANs): Fundamentos y Aplicación Adversaria en Ciberseguridad

En esta sesión, exploraremos las **Redes Generativas Antagónicas (GANs)**, un enfoque excepcionalmente poderoso en el campo del *Deep Learning* (aprendizaje profundo). Las GANs están diseñadas para generar **datos sintéticos** que son prácticamente **indistinguibles de los reales**.

Desde su introducción por Ian Goodfellow y sus colegas en 2014, las GANs han revolucionado áreas como la creación de imágenes, la síntesis de voz y la generación de texto, permitiendo avances notables en la creación de contenido artificial realista.

1. Arquitectura y Principio del Aprendizaje Adversario

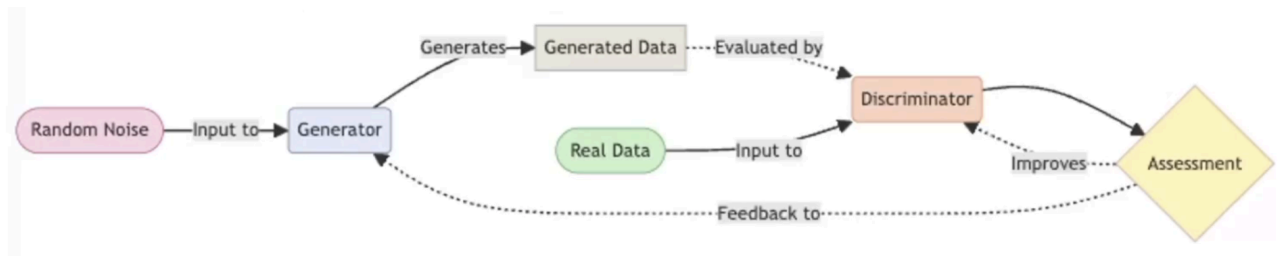
Las GANs constan de dos componentes principales que operan en un ciclo de competencia constante: el **Generador** y el **Discriminador**.

- **Generador:** Tiene la tarea de **crear datos** que imiten a los reales.
- **Discriminador:** Es responsable de **evaluar estos datos**, intentando distinguir entre los datos generados (falsos) y los auténticos (reales).

Durante el entrenamiento, estas dos redes entran en una especie de **juego de estrategia o aprendizaje adversario**. El generador aprende gradualmente a producir datos más convincentes, y el discriminador, a su vez, mejora su capacidad para detectar falsificaciones. El éxito de una red mejora al oponente, llevando a mejoras graduales en ambas.

Explicación Detallada del Flujo de una GAN

El siguiente esquema ilustra el funcionamiento de una Red Generativa Antagónica, donde el proceso de generación, evaluación y ajuste se repite en numerosas iteraciones:



1. **Ruido Aleatorio (*Random Noise*):** El proceso comienza con una serie de **valores random** o aleatorios. Este ruido (representado en color rosa) es la entrada inicial que el **Generador** utiliza como fuente de inspiración para crear algo nuevo.
2. **Generador (*Generator*):** Este componente (en color celeste o cian) toma el ruido aleatorio como entrada y, a través de su proceso interno (generalmente una red neuronal profunda), **genera nuevos datos**. Estos datos son un intento de imitar los datos reales.
3. **Datos Generados (*Generated Data*):** Son el resultado del proceso de generación (bloque gris). Aunque inicialmente pueden ser poco realistas, la meta es que con suficiente entrenamiento sean **indistinguibles de los auténticos** por el discriminador.
4. **Datos Reales (*Real Data*):** Son los **ejemplos genuinos** (bloque verde) que se usan como forma de enseñanza para el Discriminador. Estos datos se contrastan con los generados para entrenar al Discriminador en la identificación de características que los diferencian de los falsos.
5. **Discriminador (*Discriminator*):** Este componente (en color naranja) evalúa tanto los datos reales como los generados. Su trabajo es **clasificar los datos que recibe**, indicando si cree que son auténticos (reales) o falsos (generados por el Generador). El Discriminador se vuelve más hábil con el tiempo gracias a la retroalimentación continua sobre su desempeño.

6. **Evaluación (Assessment):** Es la fase (rombo amarillo) donde el Discriminador ha emitido su juicio sobre los datos. Esta evaluación determina qué tan bien está haciendo su trabajo el Generador y **cómo debe ajustarse para mejorar**. La retroalimentación del proceso de evaluación se dirige **de vuelta al Generador**, indicándole cómo ajustar su proceso de generación para que sus datos sean menos distinguibles de los reales por el Discriminador.

Este ciclo de generación, evaluación y ajuste se repite en muchas iteraciones, con el Generador y el Discriminador mejorando gradualmente su desempeño a través de este proceso adversario.

2. Implicaciones y Conclusiones en Ciberseguridad

Las GANs han tenido un impacto muy grande en el campo de la ciberseguridad, marcando tanto avances significativos en la defensa como nuevos desafíos en el ataque.

Usos Maliciosos (Ataque)

Las GANs se pueden utilizar para fines maliciosos y constructivos.

- **Generación de Contenido Falso:** Las GANs pueden generar **contenido falso, pero altamente convincente**, como las **deep fakes**.
- **Evasión de Detección:** Pueden ser usadas para crear patrones de ataque en ciberseguridad que **evadan sistemas de detección**. El potencial para crear *malware* indetectable por sistemas tradicionales presenta un riesgo muy significativo.
- **Riesgo Estratégico:** Estos avances pueden ser utilizados por actores maliciosos para llevar a cabo campañas de desinformación, fraude y ataques cibernéticos.

Usos Constructivos (Defensa)

La defensa contra estos ataques implica mejorar los sistemas de detección.

- **Generación de Datos de Entrenamiento (*Data Augmentation*):** Las GANs pueden generar datos sintéticos (llamado *data augmentation*) que sirven para entrenar mejor los sistemas de detección.
- **Fortalecimiento de Sistemas:** Esta capacidad ha permitido crear sistemas de seguridad más **robustos y adaptativos**, capaces de generar y aprender de **ataques simulados** para mejorar la detección de amenazas y la respuesta ante incidentes.
- **Anticipación a Amenazas:** Este entrenamiento avanzado fortalece las defensas contra técnicas de ataque sofisticadas, anticipándose a las estrategias de los ciberdelincuentes.

En conclusión, las GANs potencian la innovación en la ciberseguridad, pero también exigen una **evolución paralela y constante** en las estrategias de defensa para contrarrestar su posible uso malintencionado.

Implementación práctica y Entrenamiento de Redes Generativas Antagónicas (GANs)

En esta sesión, nos centraremos en el proceso de **implementación y entrenamiento de una Red Generativa Antagónica (GAN)**. El objetivo primordial es crear una GAN utilizando **Python** y un entorno de **Jupyter Notebook** que sea capaz de replicar las imágenes del **dataset de MNIST**. El *dataset* MNIST está compuesto por miles de números escritos a mano, y al entrenar la GAN para imitar estos datos, se desarrolla un modelo capaz de generar cualquier número sintético con una apariencia muy realista.

La capacidad de replicar imágenes de manera convincente tiene una base y una implicación crucial en el mundo de la ciberseguridad, particularmente en el contexto de los **CAPTCHA**. Esta implementación práctica sirve como una **plantilla** que podría modificarse y adaptarse para **pasar un control de CAPTCHA** basado en imágenes. De hecho, el modelo tal como está diseñado inicialmente sería directamente capaz de **interpretar y replicar números** si el CAPTCHA utilizara específicamente dígitos escritos a mano.

Esta primera parte del ejercicio se enfocará en la implementación práctica en Jupyter Notebook y en el proceso de entrenamiento del modelo. En la siguiente fase, analizaremos los resultados obtenidos y exploraremos una acción alternativa de ejecución utilizando la computación en la nube.

Implementación Práctica de una GAN para Replicar MNIST

1. Configuración Inicial y Definición de Hiperparámetros

El primer paso para la implementación de la GAN que imitará el *dataset* **MNIST** (números escritos a mano) es importar las librerías necesarias y establecer las variables de configuración fundamentales.

Código de Importación y Configuración

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import time

# Basic configuration
BUFFER_SIZE = 60000
BATCH_SIZE = 256
```

Análisis Detallado del Código

1. Importación de Librerías:

- Se importa **tensorflow as tf**, que es el *framework* base para la construcción y entrenamiento de la red.
- Se importa **tensorflow.keras.layers**, que proporciona las herramientas de alto nivel para definir las capas del modelo (Generador y Discriminador).
- Se importan **matplotlib.pyplot as plt** y **time**, que se utilizarán posteriormente para la visualización de los resultados generados y para medir el tiempo del entrenamiento, respectivamente.

2. Configuración Básica de Hiperparámetros:

- **BUFFER_SIZE = 60000**: Esta variable define el tamaño del *buffer* para mezclar los datos (*shuffling*). En el contexto del entrenamiento de modelos de *Deep Learning*, especialmente con conjuntos de datos grandes como MNIST, mezclar los datos (*shuffling*) es crucial porque ayuda a **reducir el riesgo de sobreajuste (*overfitting*)** y **mejora la generalización del modelo** al asegurar que los lotes de entrenamiento sean diversos. La cantidad de 60.000 se refiere directamente al **tamaño total del conjunto de datos de entrenamiento de MNIST**, que contiene 60.000 imágenes de dígitos escritos a mano.

- **BATCH_SIZE = 256**: Esta variable define el **tamaño del lote** (*batch size*) para el entrenamiento. El tamaño del lote se refiere a la **cantidad de muestras de datos** que se alimentarán a la red en una sola interacción del algoritmo de entrenamiento. Por ejemplo, en el **Descenso del Gradiente** (*Gradient Descent*), un *batch_size* de 256 indica que 256 imágenes se procesarán y pasarán por la red antes de que se **actualicen los pesos** del modelo.

2. Carga y Preprocesamiento del Dataset MNIST

Una vez establecidos los hiperparámetros básicos, el siguiente paso crucial es cargar y preparar el conjunto de datos **MNIST** para que sus imágenes puedan ser procesadas eficientemente por la Red Generativa Antagónica (GAN).

Código de Carga y Normalización:

```
# Load and prepare the MNIST dataset
(train_images, _), (_, _) =
tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28,
28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5 # Normalize the
images to [-1, 1]

# Create a TensorFlow Dataset
train_dataset =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFE
R_SIZE).batch(BATCH_SIZE)
```

Análisis Detallado del Procedimiento

1. **Carga del Conjunto de Datos:** La primera línea del código utiliza la función **`tf.keras.datasets.mnist.load_data()`**, que es la encargada de

cargar el conjunto de datos de MNIST. Este *dataset* estándar contiene una configuración predefinida de **60.000 imágenes de entrenamiento** y **10.000 imágenes de prueba**. Cada una de estas imágenes representa un dígito escrito a mano y posee una resolución de **28×28 píxeles**. La función devuelve dos tuplas con los datos de imágenes y las etiquetas asociadas (entrenamiento y prueba). En este caso, solo se utilizan las imágenes de entrenamiento, ignorando las etiquetas asociadas.

2. **Redimensionamiento y Tipo de Dato:** El siguiente paso es el redimensionamiento de las imágenes, realizado por la línea `train_images.reshape(...)`. Este paso es fundamental para asegurar que los datos tengan la **forma adecuada** que la red neuronal espera. La red requiere imágenes con **cuatro dimensiones**: (número de imágenes, altura, anchura, canales). Se añade explícitamente una dimensión de canal (**el valor 1**) porque las imágenes están en **escala de grises**. Además, se realiza la conversión al tipo de datos **float32**, que es el formato de punto flotante preferido por TensorFlow para realizar cálculos de *Deep Learning* de manera eficiente.
3. **Normalización de Imágenes:** La tercera línea aplica el proceso de normalización: `train_images = (train_images - 127.5) / 127.5`. En este caso particular de imágenes, los valores de píxeles originales que se encuentran en el rango de **0 a 255** se reescalan a valores en el rango de **-1 a 1**. La normalización es una práctica común que facilita el entrenamiento del modelo, ya que mantiene los valores de entrada en una escala manejable y estable.
4. **Creación del Objeto `tf.data.Dataset`:** El segundo bloque de código transforma las imágenes de entrenamiento en un objeto **`tf.data.Dataset`**. Este objeto permite una **manipulación eficiente** y el procesamiento en lotes de los datos, algo crucial para el correcto funcionamiento del entrenamiento de *Deep Learning*.
5. **Mezcla de Datos (*Shuffling*):** Se aplica la opción `.shuffle(BUFFER_SIZE)`. Esta función se utiliza para **mezclar los datos** de manera aleatoria, utilizando el tamaño del *buffer* previamente definido

(60.000). Mezclar los datos es esencial en el entrenamiento de aprendizaje profundo, ya que ayuda a **minimizar el riesgo de sobreajuste** (*overfitting*).

6. **Agrupación en Lotes (*Batching*)**: Finalmente, la función

.batch(BATCH_SIZE) agrupa las imágenes en lotes de un tamaño específico, que es el *BATCH_SIZE* definido (256). Entrenar el modelo en lotes permite **aprovechar la computación en paralelo**, lo que acelera significativamente el proceso de entrenamiento.

3. Visualización de Muestras del Dataset (Función **display_mnist_examples**)

Antes de construir la arquitectura de la GAN, es útil visualizar algunas de las imágenes normalizadas para confirmar la correcta carga y preparación del *dataset*. Esta función está diseñada únicamente para mostrar una pequeña muestra del conjunto de datos MNIST.

Código de la Función de Visualización

```
# Display MNIST examples
def display_mnist_examples():
    plt.figure(figsize=(10, 10))
    for i in range(16):
        plt.subplot(4, 4, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        plt.imshow(train_images[i, :, :, 0] * 0.5 + 0.5,
cmap='gray')
    plt.show()
```

Análisis Detallado del Procedimiento

La función **display_mnist_examples()** utiliza la librería **Matplotlib** (**plt**) para generar y presentar una cuadrícula de imágenes.

1. **Inicialización del Gráfico:** La línea `plt.figure(figsize=(10, 10))` inicializa una nueva figura con un tamaño de 10×10 pulgadas, preparando el lienzo para las muestras.
2. **Bucle de Iteración:** Se utiliza un bucle `for i in range(16)` para iterar 16 veces, ya que se mostrarán 16 ejemplos en una cuadrícula de 4×4.
3. **Creación de Subgráficos:** Dentro del bucle, la línea `plt.subplot(4, 4, i+1)` crea un subgráfico individual, posicionándolo en una cuadrícula de 4 filas y 4 columnas.
4. **Limpieza del Eje:** Los comandos `plt.xticks([])`, `plt.yticks([])` y `plt.grid(False)` se encargan de **ocultar las etiquetas de los ejes** y la cuadrícula, asegurando que solo se muestren las imágenes de los dígitos de forma limpia.
5. **Visualización de la Imagen:** El comando clave `plt.imshow(...)` muestra la imagen en el subgráfico actual.
 - `train_images[i, :, :, 0]`: Selecciona la i-ésima imagen de la matriz de entrenamiento. El `0` al final accede al único canal de color (escala de grises).
 - `* 0.5 + 0.5`: Esta operación es crucial porque **desnormaliza** temporalmente la imagen. Dado que las imágenes fueron normalizadas al rango `[-1,1]` para el entrenamiento de la GAN, esta operación las revierte al rango `[0,1]`, lo cual es necesario para que Matplotlib pueda renderizar los píxeles correctamente.
 - `cmap='gray'`: Especifica que la imagen debe ser mostrada en escala de grises.
6. **Mostrar la Figura:** Finalmente, `plt.show()` renderiza la figura completa en la salida del *notebook*.

La ejecución de la función `display_mnist_examples()` (cuando se invoque posteriormente) producirá la visualización de 16 ejemplos del *dataset*.

4. Definición de la Arquitectura del Modelo Generador

Esta sección define la estructura del **Modelo Generador**, que es el componente de la GAN encargado de crear las imágenes sintéticas de los números de MNIST a partir de un vector de ruido aleatorio. Utilizamos la API **Sequential de Keras** dentro de TensorFlow.

Código del Modelo Generador

```
# Simplified Generator Model
def make_simple_generator_model():
    model = tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False,
input_shape=(100,)),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),
        layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),
padding='same', use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),
padding='same', use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),
padding='same', use_bias=False, activation='tanh')
    ])
    return model
```

Análisis Detallado Capa por Capa

La función `make_simple_generator_model()` crea un modelo secuencial (`tf.keras.Sequential`), lo que significa que sus capas se apilan linealmente.

Capa Densa de Entrada y Expansión:

```
layers.Dense(7*7*256, use_bias=False, input_shape=(100,)),
```

Propósito: Esta es la primera capa, que actúa como una capa de entrada **completamente conectada** (*fully connected*) o **densa**. Toma un **vector de ruido** de dimensión 100 como entrada (`input_shape=(100,)`).

Expansión: La capa densa expande esta entrada a un tamaño de $7 \times 7 \times 256$ unidades, preparándolo para ser remodelado.

`use_bias=False`: Indica explícitamente que esta capa **no utilizará sesgo** (*bias*).

Normalización de Lote (*Batch Normalization*):

```
layers.BatchNormalization(),
```

Propósito: Normaliza las activaciones de la capa anterior, **estabilizando el aprendizaje** al reescalar la entrada de cada capa para que tenga una media de 0 y una varianza de 1.

Función de Activación Leaky ReLU:

```
layers.LeakyReLU(),
```

Propósito: Esta función de activación permite que una **pequeña cantidad de gradiente fluya** incluso cuando la unidad no esté activa (cuando la entrada es negativa), lo que ayuda a **prevenir el problema de las "neuronas muertas"** durante el entrenamiento de redes profundas.

Remodelación (*Reshape*):

```
layers.Reshape((7, 7, 256)),
```

Propósito: Esta capa cambia la forma del tensor de salida ($7 \times 7 \times 256$) a un **formato de imagen 3D** (altura, anchura y canales). Esto es un paso preparatorio crucial para las operaciones de **Convolución Traspuesta** que siguen a continuación, ya que se introducen las dimensiones espaciales.

Capa de Convolución Traspuesta (Bloque 1):

```
layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),  
padding='same', use_bias=False),
```

```
layers.BatchNormalization(),
```

```
layers.LeakyReLU(),
```

Convolución Traspuesta: Estas capas realizan la **operación inversa de una convolución** (desconvolución), y se utilizan para **aumentar progresivamente la resolución** de las imágenes generadas.

Funcionamiento: Esta primera capa de convolución traspuesta aumenta la resolución de 7×7 a 7×7 (ya que `strides=(1, 1)`), pero reduce la profundidad de 256 a **128 canales**.

Normalización y Activación: Le siguen la **Normalización de Lote** (para estabilizar) y la función de activación **Leaky ReLU**.

Capa de Convolución Traspuesta (Bloque 2 - Aumento de Resolución):

```
layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),  
padding='same', use_bias=False),
```

```
layers.BatchNormization(),
```

```
layers.LeakyReLU(),
```

Aumento de Resolución: El parámetro **strides=(2, 2)** indica que esta capa duplicará la resolución de la imagen, pasando de 7×7 a 14×14 . Al mismo tiempo, reduce la profundidad de los canales de 128 a **64 canales**.

Capa de Convolución Traspuesta Final (Salida):

```
layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),  
padding='same', use_bias=False, activation='tanh')
```

Salida Final: Esta es la última capa de convolución traspuesta. Utiliza **strides=(2, 2)** para aumentar la resolución nuevamente, de 14×14 a **28×28** (el tamaño final de la imagen MNIST).

1 canal de salida: El primer argumento es **1**, indicando que la imagen final debe tener solo **un canal** (escala de grises).

Función de Activación Tanh: La función de activación **tangente hiperbólica** (**tanh**) se aplica a la salida. Esto **escala los valores de los píxeles al rango de $[-1, 1]$** , lo que coincide perfectamente con la normalización aplicada a las imágenes reales del *dataset* MNIST al inicio del proceso.

Relevancia en Ciberseguridad

Esta plantilla de código para la generación de imágenes tiene implicaciones directas en la ciberseguridad, especialmente en la **manipulación e imitación de imágenes**. El objetivo de haber elegido este *dataset* MNIST y esta arquitectura de GAN es crear un modelo capaz de generar un **gestor anti-CAPTCHAs** que sea capaz de **saltarse los controles de CAPTCHAs** basados en imágenes (como la interpretación de números). Esta implementación es una base que se puede

modificar y adaptar para imitar o generar otros *datasets* basados en imágenes para fines de ataque o defensa.

5. Definición de la Arquitectura del Modelo Discriminador

El **Modelo Discriminador** es el clasificador de la GAN, cuya función es **evaluar** si una imagen de entrada (generada por el Generador o extraída del *dataset* real) es **real o falsa**. Utilizamos la API Sequential de Keras en TensorFlow, al igual que hicimos con el Generador.

Código del Modelo Discriminador

```
# Simplified Discriminator Model
def make_simple_discriminator_model():
    model = tf.keras.Sequential([
        layers.Conv2D(64, (5, 5), strides=(2, 2),
padding='same', input_shape=[28, 28, 1]),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Conv2D(128, (5, 5), strides=(2, 2),
padding='same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Flatten(),
        layers.Dense(1)
    ])
    return model
```

Análisis Detallado Capa por Capa

La función `make_simple_discriminator_model()` construye un modelo secuencial (`tf.keras.Sequential`) que procesa la imagen y la clasifica.

Primera Capa de Convolución (*Conv2D*):

```
layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same',  
input_shape=[28, 28, 1]),
```

Propósito y Entrada: Esta es la capa de entrada del discriminador. Toma imágenes de tamaño 28×28 píxeles con **un solo canal** (`input_shape=[28, 28, 1]`) como entrada.

Filtros (*Filters*): Utiliza **64 filtros** con un tamaño de *kernel* (núcleo) de 5×5.

Strides: El parámetro `strides=(2, 2)` indica que el *kernel* se mueve **dos píxeles** tanto en la altura como en la anchura. Esto tiene el efecto de **reducir a la mitad las dimensiones espaciales de la salida**, lo cual es clave para el proceso de convolución. Para entender mejor este efecto, se recomienda profundizar en el funcionamiento del paso (*stride*) y del *kernel* en las redes convolucionales.

Padding: El parámetro `padding='same'` asegura que el tamaño de la salida se mantenga proporcional, ajustando adecuadamente la imagen de entrada con ceros en sus bordes si es necesario.

Activación y Regularización:

```
layers.LeakyReLU(),  
layers.Dropout(0.3),
```

Leaky ReLU: Se utiliza la función de activación **Leaky ReLU** para permitir el flujo de un pequeño gradiente, previniendo el problema de las neuronas inactivas.

Dropout: El **Dropout(0.3)** aplica una técnica de regularización.

Aleatoriamente, el 30% de las características de entrada se ponen a cero en cada actualización durante el entrenamiento, lo cual es una técnica muy específica utilizada para **prevenir el sobreajuste** (*overfitting*).

Segunda Capa de Convolución y Regularización:

```
layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'),  
layers.LeakyReLU(),  
layers.Dropout(0.3),
```

Aumento de Filtros: Esta capa es similar a la primera, pero **aumenta la cantidad de filtros** a 128. Continúa reduciendo las dimensiones espaciales de la imagen de entrada a la mitad, mientras extrae características más complejas. La combinación de **Leaky ReLU** y **Dropout** se repite para mantener la estabilidad y evitar el sobreajuste.

Aplanamiento (*Flatten*):

```
layers.Flatten(),
```

Propósito: Las capas convolucionales y de *pooling* dejan el tensor de entrada en un formato 3D. El comando **Flatten()** es esencial porque **aplasta el tensor de entrada** para convertirlo en un único **vector 1D**. Este vector unidimensional es el formato requerido por la capa densa final.

Capa Densa de Salida:

```
layers.Dense(1)
```


Propósito: Esta capa densa, con **solo una unidad (1)**, produce la salida del discriminador. Esta salida es una **predicción escalar**, representando la **probabilidad** de que la imagen de entrada sea real y no generada.

Activación (Implícita): Aunque no se aplica ninguna activación específica directamente en la capa, típicamente se utilizaría una función **sigmoide** posteriormente (durante la compilación del modelo) para obtener una probabilidad entre 0 y 1, especialmente si se usa una pérdida de entropía cruzada binaria.

6. Definición de la Función de Pérdida y los Optimizadores

Esta sección establece las **funciones de pérdida** (*loss functions*) y los **optimizadores** que guiarán el proceso de aprendizaje adversario entre el Generador y el Discriminador.

Código de Configuración de Pérdida y Optimizadores

```
# Loss function and optimizers
cross_entropy =
tf.keras.losses.BinaryCrossentropy(from_logits=True)

generator_optimizer = tf.optimizers.Adam(1e-4)
discriminator_optimizer = tf.optimizers.Adam(1e-4)
```

Análisis Detallado del Procedimiento

1. Definición de la Función de Pérdida (**cross_entropy**):

- La línea `cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)` crea una función de pérdida de **entropía cruzada binaria** (`BinaryCrossentropy`), que es la función estándar utilizada en problemas de clasificación binaria (real vs. falso).
- El parámetro crucial aquí es **from_logits=True**. Este parámetro indica que las salidas del Discriminador se interpretarán como **"logits"**, que son valores de salida que **no han pasado** por la función

de activación sigmoide (la función que los convertiría en probabilidades). Esta configuración se utiliza cuando el modelo (`layers.Dense(1)` en el Discriminador) no tiene la función sigmoide aplicada al final.

2. Definición del Optimizador del Generador (`generator_optimizer`):

- La línea `generator_optimizer = tf.optimizers.Adam(1e-4)` define el optimizador que ajustará los pesos del Generador.
- Se utiliza el algoritmo **Adam**, uno de los optimizadores más comunes y eficientes.
- Se establece una **tasa de aprendizaje** de **0.0001** (`1e-4`).

3. Definición del Optimizador del Discriminador

(`discriminator_optimizer`):

- De manera similar, la línea `discriminator_optimizer = tf.optimizers.Adam(1e-4)` define el optimizador para el Discriminador, utilizando también el algoritmo **Adam** con la misma tasa de aprendizaje (0.0001).

La Necesidad de Optimizadores Separados

Es fundamental tener **optimizadores separados** para el Generador y el Discriminador porque cada modelo se entrena individualmente en un proceso de dos pasos durante el entrenamiento de una GAN:

- **Generador (G):** Intenta **minimizar** la capacidad del Discriminador para distinguir entre imágenes reales y generadas, reduciendo así su función de pérdida.
- **Discriminador (D):** Intenta **maximizar** su propia capacidad para distinguir correctamente las imágenes reales de las falsas, aumentando así su función de pérdida.

En resumen, esta sección establece el marco necesario para que la GAN pueda calcular el desempeño de ambos modelos y ajustar sus pesos mediante la **Retropropagación**, asegurando que ambos mejoren continuamente a lo largo del proceso adversario.

7. Definición del Ciclo de Entrenamiento: Función `train_step`

La función `train_step` encapsula el ciclo de aprendizaje adversario entre el Generador y el Discriminador, asegurando que los pesos de ambos modelos se ajusten correctamente mediante la **Retropropagación**.

Código de la Función `train_step`

```
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images,
training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output,
fake_output)

        gradients_of_generator = gen_tape.gradient(gen_loss,
generator.trainable_variables)
        gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator
, generator.trainable_variables))

    discriminator_optimizer.apply_gradients(zip(gradients_of_discr
iminators, discriminator.trainable_variables))

    return gen_loss, disc_loss
```

Análisis Detallado del Procedimiento:

Aceleración con el Decorador `@tf.function`:

La función comienza con el decorador `@tf.function`. Este es un elemento nativo de TensorFlow muy útil que convierte la función Python `train_step` en una **función TensorFlow compilada**.

Propósito: Esta compilación mejora significativamente el **rendimiento** al permitir que TensorFlow optimice automáticamente el cálculo, ejecutando el entrenamiento como un **grafo estático** de operaciones en lugar de operaciones discretas de Python. Esto es una práctica muy recomendada para acelerar el entrenamiento en modelos de *Deep Learning* que contienen operaciones complejas y repetitivas.

Generación de Ruido de Entrada:

```
noise = tf.random.normal([BATCH_SIZE, 100])
```

Propósito: Esta línea genera un tensor de **ruido aleatorio**, normalmente distribuido, con la forma `[BATCH_SIZE, 100]`. Este ruido sirve como **entrada** para el Generador, permitiéndole producir imágenes nuevas y variadas.

Apertura de las Cintas de Gradiente (*Gradient Tapes*):

```
with tf.GradientTape() as gen_tape, tf.GradientTape() as  
disc_tape:  
  
    # Bloque de operaciones que se registrarán  
  
    # ...
```

Mecanismo: Se inicializan dos contextos de `tf.GradientTape`: uno para el Generador (`gen_tape`) y otro para el Discriminador (`disc_tape`).

Propósito: TensorFlow utiliza estas cintas para **registrar todas las operaciones** necesarias para la diferenciación automática. Esto es crucial porque permite calcular los gradientes de la función de pérdida con respecto a las variables entrenables de ambos modelos por separado.

Generación de Imágenes y Predicciones (Dentro del Contexto `with`):

Generación de Imágenes: `generated_images = generator(noise, training=True)` utiliza el Generador para crear imágenes a partir del ruido generado.

Predicción de Imágenes Reales: `real_output = discriminator(images, training=True)` pasa las **imágenes reales** al Discriminador y obtiene sus predicciones de la realidad.

Predicción de Imágenes Falsas: `fake_output = discriminator(generated_images, training=True)` pasa las **imágenes generadas** al Discriminador y obtiene sus predicciones de "falsedad".

Cálculo de Pérdidas (*Loss Calculation*):

Pérdida del Generador: `gen_loss = generator_loss(fake_output)` calcula la pérdida del Generador basándose en **cómo el Discriminador evalúa las imágenes generadas**. El Generador quiere que estas imágenes sean clasificadas como **reales**.

Pérdida del Discriminador: `disc_loss = discriminator_loss(real_output, fake_output)` calcula la pérdida del Discriminador basándose en su rendimiento al evaluar **tanto las imágenes reales como las generadas**.

Cálculo y Aplicación de Gradientes (Fuera del Contexto `with`):

Cálculo de Gradientes: Los gradientes se calculan utilizando las cintas grabadas:

```
gradients_of_generator = gen_tape.gradient(gen_loss,  
generator.trainable_variables)  
  
gradients_of_discriminator = disc_tape.gradient(disc_loss,  
discriminator.trainable_variables)
```

Este paso utiliza la información registrada para calcular cómo debe cambiar cada peso en el Generador y el Discriminador para minimizar sus respectivas pérdidas.

Aplicación de Gradientes: Finalmente, los optimizadores definidos aplican los gradientes calculados para actualizar los pesos de cada modelo:

```
generator_optimizer.apply_gradients(zip(gradients_of_generator  
, generator.trainable_variables))  
  
discriminator_optimizer.apply_gradients(zip(gradients_of_discr  
iminator, discriminator.trainable_variables))
```

Este proceso se realiza con optimizadores separados, ya que cada modelo se entrena individualmente. Mientras el Generador intenta minimizar la pérdida (engañar al Discriminador), el Discriminador intenta maximizar su pérdida (clasificar correctamente).

La función devuelve ambas pérdidas (`gen_loss`, `disc_loss`) para el monitoreo del entrenamiento.

8. Definición de las Funciones de Pérdida para la GAN

Estas dos funciones son cruciales porque proporcionan el mecanismo para cuantificar el error de cada modelo, permitiendo así que el **Descenso de Gradiente** (aplicado en `train_step`) ajuste sus parámetros.

Código de las Funciones de Pérdida

```
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output),
                               real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output),
                               fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output),
                          fake_output)
```

Análisis Detallado del Procedimiento

Estas funciones utilizan la pérdida de **entropía cruzada binaria** (`cross_entropy`) que definimos previamente, la cual opera con *logits* (valores de salida sin activación sigmoide).

1. Función de Pérdida del Discriminador (**discriminator_loss**):

- Esta función toma como entradas las predicciones del Discriminador sobre las **imágenes reales** (`real_output`) y las predicciones sobre las **imágenes generadas** (`fake_output`). Su objetivo es **maximizar** la capacidad de clasificación del Discriminador (es decir, acercar su predicción a la etiqueta correcta).
- **Pérdida con Datos Reales (`real_loss`):** `real_loss = cross_entropy(tf.ones_like(real_output), real_output)` calcula la pérdida asumiendo que las imágenes reales son "1" (`tf.ones_like`), que es la etiqueta correcta.

- **Pérdida con Datos Falsos (`fake_loss`):** `fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)` calcula la pérdida asumiendo que las imágenes generadas son "0" (`tf.zeros_like`), que también es la etiqueta correcta para el Discriminador.
- **Pérdida Total:** `total_loss = real_loss + fake_loss` simplemente suma las dos pérdidas. El Discriminador busca minimizar esta pérdida total, lo que significa que está mejorando su precisión al clasificar tanto las imágenes reales como las falsas.

2. Función de Pérdida del Generador (`generator_loss`):

- Esta función solo toma como entrada las predicciones del Discriminador sobre las **imágenes generadas** (`fake_output`).
- **Propósito:** El Generador tiene la tarea adversaria de **engañar** al Discriminador. Por lo tanto, calcula su pérdida asumiendo que las imágenes generadas deberían ser clasificadas como "1" (`tf.ones_like`), ya que quiere que el Discriminador crea que las imágenes falsas son reales.
- La pérdida del Generador es alta si el Discriminador es bueno (clasifica la imagen como 0) y baja si el Generador es bueno (el Discriminador clasifica la imagen incorrectamente como 1). Al **minimizar esta pérdida**, el Generador mejora su capacidad de producir imágenes realistas.

Ambas funciones, la `discriminator_loss` y la `generator_loss`, fueron esenciales para el cálculo de los gradientes en el ciclo de entrenamiento `train_step`.

9. Definición de la Fase de Entrenamiento Principal: Función `train`

La función `train` define el bucle de entrenamiento completo para la GAN, iterando a través del conjunto de datos durante múltiples épocas (*epochs*) y orquestando la actualización de los modelos **Generador** y **Discriminador** en cada paso.

Código de la Función `train`

```
def train(dataset, epochs):
    for epoch in range(epochs):
        start = time.time()

        for batch, image_batch in enumerate(dataset):
            gen_loss, disc_loss = train_step(image_batch)
            if batch % 100 == 0: # Adjust the logging
                frequency according to your dataset size
                print(f'Epoch: {epoch+1}, Batch: {batch}, Gen
                    Loss: {gen_loss}, Disc Loss: {disc_loss}')

        print(f'Epoch {epoch+1} completed in
            {time.time()-start:.2f} seconds')
```

Análisis Detallado del Procedimiento:

La función `train` toma como argumentos el conjunto de datos preparado (`dataset`) y el número de épocas deseadas (`epochs`).

Bucle de Épocas (*Epochs*):

```
for epoch in range(epochs):
```

Propósito: El bucle más externo se encarga de iterar a través del número total de épocas de entrenamiento. Una época representa un ciclo completo en el que la red ha visto todo el conjunto de datos.

Registro del Tiempo de Inicio:

```
start = time.time()
```

Propósito: Este paso calcula y almacena el tiempo de inicio de la época. Esto se realiza para calcular y mostrar posteriormente **cuánto tiempo toma completar cada época**, lo cual es útil para monitorizar la evolución del entrenamiento y su eficiencia.

Bucle de Lotes (*Batches*):

```
for batch, image_batch in enumerate(dataset):  
  
    # ...
```

Propósito: Esta iteración va a través de los lotes del conjunto de datos dentro de cada época. Se itera sobre el *dataset*, el cual ya ha sido dividido en lotes previamente.

Ejecución del Paso de Entrenamiento:

```
gen_loss, disc_loss = train_step(image_batch)
```

Propósito: Para cada lote de imágenes (*image_batch*), se llama a la función **train_step**. Esta función realiza el **paso de entrenamiento completo** (generación, predicción, cálculo de gradientes y actualización de pesos), devolviendo la pérdida del Generador (*gen_loss*) y la pérdida del Discriminador (*disc_loss*).

Registro de Pérdidas y Progreso:

```
if batch % 100 == 0:

    print(f'Epoch: {epoch+1}, Batch: {batch}, Gen Loss:
{gen_loss}, Disc Loss: {disc_loss}')
```

Propósito: Se incluye una condición para **registrar el progreso** de la pérdida solo cada 100 lotes (`batch % 100 == 0`). Esto permite monitorizar cómo va evolucionando el entrenamiento sin saturar la pantalla con *outputs* innecesarios, lo cual es esencial para grandes *datasets*.

Registro del Tiempo de Finalización:

```
print(f'Epoch {epoch+1} completed in {time.time()-start:.2f}
seconds')
```

Propósito: Al finalizar el bucle de lotes, se calcula la diferencia entre el tiempo actual y el tiempo de inicio (`start`), proporcionando un registro del tiempo total que ha tomado completar la época.

10. Visualización de Resultados: Función `visualize_results`

Esta función está diseñada para **visualizar las imágenes sintéticas** generadas por el Generador de la GAN una vez que el proceso de entrenamiento ha finalizado.

Código de la Función de Visualización

```
# Function to visualize results after training
def visualize_results(model, num_examples=16):
    noise = tf.random.normal([num_examples, 100])
    generated_images = model(noise, training=False)

    plt.figure(figsize=(10, 10))
    for i in range(num_examples):
```

```
plt.subplot(4, 4, i+1)
plt.imshow(generated_images[i, :, :, 0] * 0.5 + 0.5,
cmap='gray')
plt.axis('off')
plt.show()
```

Análisis Detallado del Procedimiento

La función `visualize_results` toma como entrada el modelo Generador entrenado (`model`) y, opcionalmente, el número de imágenes de ejemplo a generar (`num_examples`, por defecto 16).

Generación de Ruido:

```
noise = tf.random.normal([num_examples, 100])
```

Propósito: Se crea un nuevo conjunto de **ruido aleatorio** (un tensor) para alimentar al Generador. Este *input* de ruido es la semilla para crear nuevas imágenes.

Generación de Imágenes:

```
generated_images = model(noise, training=False)
```

Propósito: El Generador (`model` en este contexto) utiliza el ruido para producir imágenes sintéticas. Es importante destacar que el parámetro **`training=False`** se establece para garantizar que el modelo opere en modo de inferencia y no afecte las estadísticas de normalización por lotes (*Batch Normalization*).

Configuración de la Figura y Visualización:

- **`plt.figure(figsize=(10, 10))`**: Se inicializa la figura para mostrar los resultados.

- **Bucle y Subgráficos:** Se itera sobre el número de ejemplos. Dentro del bucle, `plt.subplot(4, 4, i+1)` crea la cuadrícula de 4×4.
- **Desnormalización y Renderizado:** La línea `plt.imshow(...)` muestra la imagen generada. Al igual que en el análisis inicial del *dataset*, la operación `* 0.5 + 0.5` es esencial para **desnormalizar** los valores de píxeles del rango $[-1,1]$ (la salida del Generador) de vuelta al rango visualizable de $[0,1]$. Se usa `cmap='gray'` para la escala de grises.
- **Limpieza Visual:** `plt.axis('off')` oculta los ejes y las marcas para una presentación más limpia de la imagen.

Mostrar Resultados: Finalmente, `plt.show()` presenta la cuadrícula de imágenes sintéticas generadas por la GAN.

11. Referencia Visual del Dataset MNIST

Es importante tener una referencia visual del *dataset* original para poder juzgar el rendimiento de la GAN. Los ejemplos de MNIST son **números escritos a mano**, donde cada dígito muestra variaciones en la caligrafía. El objetivo del Generador es imitar con éxito estas diferentes variaciones, tal como se puede apreciar en las muestras del *dataset* original.

12. Ejecución y Monitorización del Entrenamiento de la GAN

La fase final del procedimiento es iniciar el entrenamiento de la GAN, alimentando la función `train` con el *dataset* de MNIST previamente preparado durante un número específico de épocas.

Código de Entrenamiento

```
# Train the model
EPOCHS = 50
train(train_dataset, EPOCHS)
```

Análisis del Procedimiento de Entrenamiento

1. **Definición y Ejecución:** La variable **EPOCHS** se define con el valor de **50**, estableciendo el número total de ciclos completos en los que el modelo verá el conjunto de datos. La función **train(train_dataset, EPOCHS)** es invocada, pasando el *dataset* de entrenamiento y el número de épocas.
2. **Proceso de Entrenamiento:** El proceso comienza la ejecución, y se espera que tarde un tiempo considerable en completarse. Durante el entrenamiento, la función **train** llama repetidamente a **train_step**, ajustando los pesos del **Generador** y el **Discriminador** en el ciclo adversario.

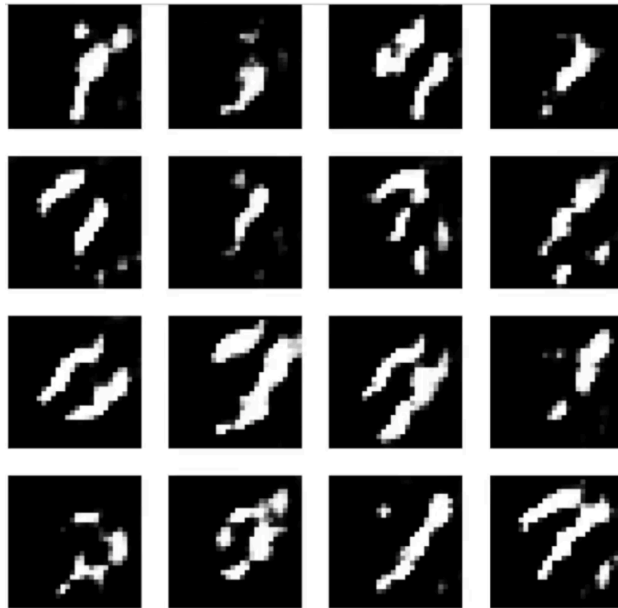
Monitorización Intermedia y Visualización de Resultados

Para monitorizar el progreso antes de que se completen las 50 épocas, el entrenamiento puede ser interrumpido intencionalmente (por ejemplo, en la época 12, generando un error de *Key Interrupt*). Tras la interrupción, se utiliza la función de visualización para inspeccionar la calidad de las imágenes generadas hasta ese momento.

- **Visualización de Imágenes Generadas (Punto Intermedio):** La función **visualize_results(generator)** se llama, utilizando el Generador en su estado de entrenamiento intermedio.

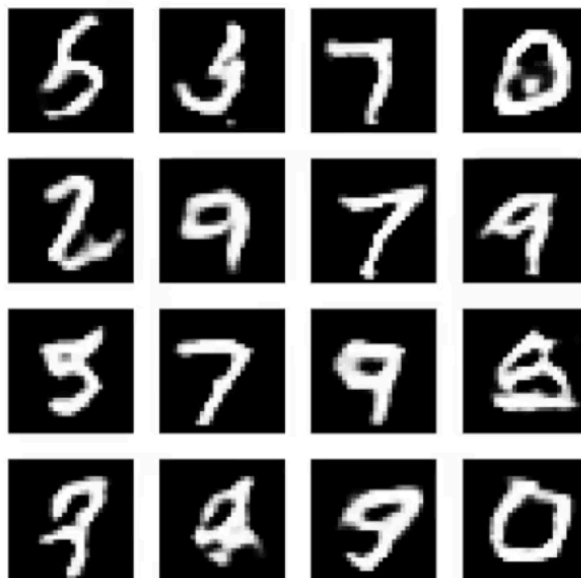
```
# Visualize generated images  
visualize_results(generator)
```

- **Inspección del Aprendizaje:** La visualización de las imágenes generadas en un punto intermedio (como la época 12) muestra que el Generador **ya está empezando a entender y buscar formas**. Aunque las imágenes aún no son números perfectamente definidos, el Generador está **atisbando la solución** y va dibujando patrones. Esto confirma que el modelo está aprendiendo poco a poco a simular las formas de los dígitos.



13. Conclusión y Ejecución Final

El resultado final, tras el entrenamiento completo, demuestra la capacidad de la GAN para **replicar las imágenes de los números escritos a mano de MNIST**, confirmando la utilidad de esta plantilla para la imitación de imágenes.



14. Ejecución en la Nube y Análisis Final de la GAN

La finalización del entrenamiento de la Red Generativa Antagónica (GAN) se realiza aprovechando la potencia de cómputo en la nube. Esta sección cubre las ventajas de este enfoque, la interpretación de los resultados tras **50 épocas**, y las conclusiones clave sobre la utilidad de las GANs en ciberseguridad.

A. Ventajas de la Ejecución en Google Colab

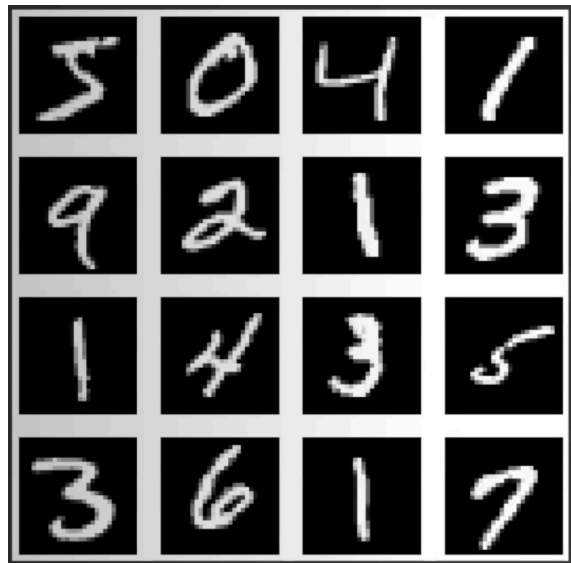
Para superar los largos tiempos de entrenamiento en entornos locales (donde 50 épocas podrían llevar horas), se traslada el código de **Jupyter Notebook** a **Google Colab**.

- **Proceso de Uso:** Para utilizar Google Colab, solo se necesita una cuenta de Google. Funciona de manera idéntica al entorno Jupyter Notebook local.
- **Aprovechamiento de Hardware:** La principal ventaja es la posibilidad de utilizar el *Runtime* (entorno de ejecución) de Colab para acceder a hardware de alta potencia, como **GPUs (Graphics Processing Units)** o **TPUs (Tensor Processing Units)**, que Google ofrece incluso a nivel gratuito.
- **Ahorro de Tiempo:** Al activar la GPU (ej., una T4 GPU), el tiempo de entrenamiento se reduce drásticamente. En este caso, el entrenamiento completo hasta las **50 épocas** se completa en aproximadamente **15 minutos**, un tiempo que habría sido inviable en un equipo local estándar.
- **Características Adicionales:** Colab también ofrece opciones más profesionales, como la conexión con **Google Drive** (que funciona creando un pequeño directorio de máquina virtual Linux al que se suben los ficheros) para subir y administrar *datasets*, facilitando la experimentación. Existen otras alternativas de servicios en la nube (más allá de Google Colab) que permiten entrenar modelos de forma gratuita o a un precio módico, ofreciendo el mismo beneficio de computación.

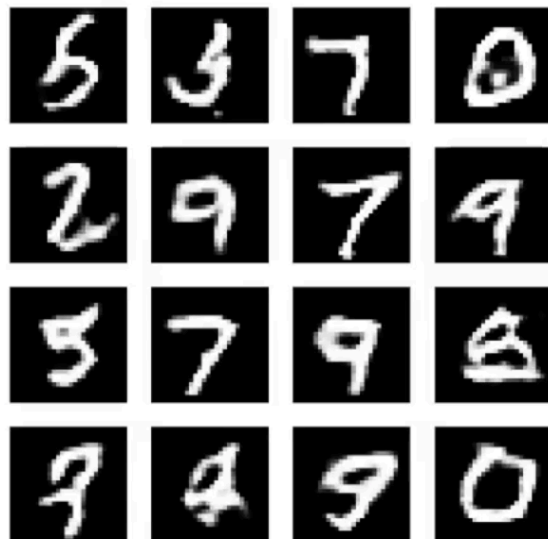
B. Análisis Visual de la Generación de Dígitos (Época 50)

El objetivo de la GAN era replicar el **dataset de MNIST** (números escritos a mano). Tras 50 épocas, la comparación visual de los resultados es la mejor forma de evaluar la progresión del Generador.

1. **Imágenes Originales (MNIST):** El *dataset* original muestra dígitos claros, escritos a mano, con variaciones naturales en la caligrafía.



2. **Imágenes Generadas (Época 50):** Las imágenes sintéticas generadas por la GAN en la época 50 demuestran un éxito considerable. Aunque la perfección puede requerir más épocas o un ajuste de hiperparámetros, el modelo ha logrado:
 - **Imitar las Formas:** La GAN ha aprendido las características estructurales de los dígitos, imitando números como el 2, el 9, el 7, el 1, y el 0.
 - **Generalización:** Las formas generadas confirman que la GAN está utilizando los ciclos adversarios entre el Generador y el Discriminador para acercarse lo máximo posible al estilo original de escritura a mano.



C. Interpretación de las Métricas de Pérdida (Época 50)

Aunque la comparación visual es el indicador más claro en este caso, es fundamental entender los **valores numéricos** que monitorizan el proceso de aprendizaje:

Epoch: 50, Batch: 0, Gen Loss: 0.8650115132331848, Disc Loss: 1.280616998672485

Epoch: 50, Batch: 100, Gen Loss: 0.9785632491111755, Disc Loss: 1.120623860168457

Epoch: 50, Batch: 200, Gen Loss: 1.197367787361145, Disc Loss: 1.1310126820999146

1. Pérdida del Generador (*Gen Loss*):

- **Significado:** Indica lo bien que el Generador está **engañando al Discriminador**. Una pérdida más baja sugiere que el Generador está produciendo imágenes que el Discriminador tiene dificultades para distinguir de las reales.
- **Análisis:** Los valores observados muestran cierta variación, pero indican que el Generador está mejorando su capacidad de generar imágenes auténticas.

2. Pérdida del Discriminador (*Disc Loss*):

- **Significado:** Refleja lo bien que el Discriminador está **clasificando correctamente** las imágenes reales y generadas. Una pérdida más alta indica que el Discriminador está teniendo dificultades para distinguir entre imágenes reales y generadas (lo cual es el objetivo del Generador).
- **Análisis:** La variabilidad de la pérdida del Discriminador sugiere que está siendo constantemente **desafiado** por las imágenes que se están generando de forma mejorada.

El hecho de que, en la época 50, las pérdidas sigan mostrando una variación (es decir, **la competencia está activa**) es un **signo positivo** de que el entrenamiento está progresando como se espera de una GAN. Idealmente, ambos modelos mejoran juntos en un equilibrio dinámico. Si se dejara más tiempo, o si se ajustaran algunos hiperparámetros (como la tasa de aprendizaje o el tipo de función de pérdida), el Generador posiblemente alcanzaría una perfección mayor.

D. Conclusiones Finales

El uso de GANs demuestra su capacidad para replicar de manera realista los dígitos de MNIST, lo que sienta las bases para posibles aplicaciones directas en ciberseguridad:

- **Evasión de CAPTCHA:** Este estudio se centra en la importancia de aprovechar las GANs para abordar desafíos relacionados con el reconocimiento de imágenes y la autenticación. La plantilla puede modificarse para **eludir sistemas CAPTCHA** basados en imágenes.
- **Seguridad Informática:** La capacidad de generar datos nuevos y realistas mediante GANs abre muchísimas posibilidades en la seguridad informática, donde la **detección y autenticación de imágenes** son fundamentales para la protección del sistema y la prevención de fraudes.
- **Eficiencia en la Nube:** Hemos visto que uno de los mayores problemas, el **tiempo de entrenamiento**, se combate eficazmente con la ejecución en la nube (como Google Colab), lo que permite un ahorro considerable de tiempo

y recursos de cómputo, acelerando el desarrollo y la optimización de modelos.

Bloque de código entero del modelo:

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import time

# Basic configuration
BUFFER_SIZE = 60000
BATCH_SIZE = 256

# Load and prepare the MNIST dataset
(train_images, _), (_, _) =
tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28,
28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5 # Normalize the
images to [-1, 1]

# Create a TensorFlow Dataset
train_dataset =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

# Display MNIST examples
def display_mnist_examples():
    plt.figure(figsize=(10, 10))
    for i in range(16):
        plt.subplot(4, 4, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
```

```

        plt.imshow(train_images[i, :, :, 0] * 0.5 + 0.5,
cmap='gray')
        plt.show()

# Simplified Generator Model
def make_simple_generator_model():
    model = tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False,
input_shape=(100,)),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((7, 7, 256)),

        layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),
padding='same', use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),

        layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),
padding='same', use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),

        layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),
padding='same', use_bias=False, activation='tanh')
    ])
    return model

# Simplified Discriminator Model
def make_simple_discriminator_model():
    model = tf.keras.Sequential([
        layers.Conv2D(64, (5, 5), strides=(2, 2),
padding='same', input_shape=[28, 28, 1]),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Conv2D(128, (5, 5), strides=(2, 2),
padding='same'),

```

```

        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Flatten(),
        layers.Dense(1)
    ])
    return model

generator = make_simple_generator_model()
discriminator = make_simple_discriminator_model()

# Loss function and optimizers
cross_entropy =
tf.keras.losses.BinaryCrossentropy(from_logits=True)

generator_optimizer = tf.optimizers.Adam(1e-4)
discriminator_optimizer = tf.optimizers.Adam(1e-4)

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as
disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images,
training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output,
fake_output)

        gradients_of_generator = gen_tape.gradient(gen_loss,
generator.trainable_variables)
        gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

```

```
generator_optimizer.apply_gradients(zip(gradients_of_generator
, generator.trainable_variables))
```

```
discriminator_optimizer.apply_gradients(zip(gradients_of_discr
iminators, discriminator.trainable_variables))
```

```
    return gen_loss, disc_loss
```

```
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output),
real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output),
fake_output)
    total_loss = real_loss + fake_loss
    return total_loss
```

```
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output),
fake_output)
```

```
def train(dataset, epochs):
    for epoch in range(epochs):
        start = time.time()

        for batch, image_batch in enumerate(dataset):
            gen_loss, disc_loss = train_step(image_batch)
            if batch % 100 == 0: # Adjust the logging
frequency according to your dataset size
                print(f'Epoch: {epoch+1}, Batch: {batch}, Gen
Loss: {gen_loss}, Disc Loss: {disc_loss}')
```

```
        print(f'Epoch {epoch+1} completed in
{time.time()-start:.2f} seconds')
```

```
# Function to visualize results after training
def visualize_results(model, num_examples=16):
```

```
noise = tf.random.normal([num_examples, 100])
generated_images = model(noise, training=False)

plt.figure(figsize=(10, 10))
for i in range(num_examples):
    plt.subplot(4, 4, i+1)
    plt.imshow(generated_images[i, :, :, 0] * 0.5 + 0.5,
cmap='gray')
    plt.axis('off')
plt.show()

# Show MNIST examples
display_mnist_examples()

# Train the model
EPOCHS = 50
train(train_dataset, EPOCHS)

# Visualize generated images
visualize_results(generator)
```